

# Implementação do Protocolo Diffie-Hellman e Esquema ElGamal

Trabalho 1 — Segurança Computacional

*Gustavo Choueiri – 232014010*  
*Universidade de Brasília*

## 1. Descrição da Solução

O trabalho implementa, em Python puro, um sistema completo de criptografia assimétrica baseado no protocolo Diffie-Hellman para estabelecimento de parâmetros e no esquema ElGamal para cifração e decifração de mensagens. Todo o código foi desenvolvido sem uso de bibliotecas criptográficas externas — as únicas importações são `secrets` (para geração de números aleatórios criptograficamente seguros), `math` e `sys` (utilitários padrão).

A simulação reproduz uma comunicação entre dois participantes, A e B, por chamadas de função e entrada/saída padrão. O fluxo segue exatamente o especificado no enunciado: A gera os parâmetros públicos  $p$  e  $g$ , B gera seu par de chaves, A cifra a mensagem com a chave pública de B e, por fim, B recupera o texto original usando sua chave privada.

Os principais módulos implementados são:

- **Geração de primos seguros (primo de Sophie Germain):** primos  $p = 2q + 1$  onde  $q$  também é primo, garantindo estrutura de grupo adequada para o ElGamal.
- **Teste de primalidade Miller-Rabin:** implementado manualmente com 10 rodadas de testemunhas aleatórias.
- **Exponenciação modular eficiente:** algoritmo square-and-multiply (exponenciação binária).
- **Inverso modular:** via algoritmo de Euclides estendido.
- **Raiz primitiva:** encontrada pelo método de verificação com fatoração de  $p-1$ .
- **Cifração/decifração ElGamal:** com suporte a mensagens arbitrárias divididas em blocos.

## 2. Implementação do Diffie-Hellman

O protocolo Diffie-Hellman define os parâmetros públicos sobre os quais o ElGamal opera. São necessários um número primo grande  $p$  e um gerador  $g$  do grupo multiplicativo  $\mathbb{Z}^*_p$ .

### 2.1 Geração do primo $p$

Foi adotada a estratégia de primos seguros (safe primes): gera-se um primo  $q$  de 255 bits e verifica-se se  $p = 2q + 1$  também é primo. Esse formato é chamado de primo de Sophie Germain e garante que a ordem do grupo seja  $2q$ , cujos únicos fatores primos são 2 e  $q$  — propriedade essencial para a geração eficiente da raiz primitiva.

A função `gerar_primo_seguro()` repete a tentativa até encontrar um par  $(q, p)$  válido. Como o teste Miller-Rabin é probabilístico, a geração pode levar algumas iterações, mas converge rapidamente na prática com chaves de 256 bits.

### 2.2 Geração do gerador $g$

A função `achar_raiz_primitiva(p, factors)` busca o menor inteiro  $g \geq 2$  que satisfaça a condição de ser gerador do grupo. Como  $p-1 = 2q$ , basta verificar duas condições:

- $g^{(p-1)/2} \not\equiv 1 \pmod{p}$
- $g^{(p-1)/q} \not\equiv 1 \pmod{p}$

Se ambas forem satisfeitas,  $g$  é uma raiz primitiva módulo  $p$ , conforme o método descrito no enunciado.

### 3. Implementação do ElGamal

O esquema ElGamal é um sistema de criptografia de chave pública baseado no problema do logaritmo discreto. Sua segurança depende diretamente da dificuldade de calcular  $x$  dado  $g^x \bmod p$ .

#### 3.1 Geração de chaves

O participante B escolhe um inteiro secreto  $x$  aleatório no intervalo  $[2, p-2]$  como chave privada, e calcula a chave pública  $y = g^x \bmod p$ . O par  $(p, g, y)$  é compartilhado publicamente;  $x$  permanece secreto.

#### 3.2 Cifração ( $A \rightarrow B$ )

Para cifrar, A escolhe um inteiro efêmero  $k$  aleatório e calcula:

- $c1 = g^k \bmod p$
- $c2 = m \cdot y^k \bmod p$  (para cada bloco  $m$  da mensagem)

O texto cifrado enviado de A para B é o par  $(c1, [c2_1, c2_2, \dots])$ . O valor  $k$  é descartado após o uso.

#### 3.3 Decifração (B)

B calcula o segredo compartilhado  $s = c1^x \bmod p = g^{(kx)} \bmod p$ , que coincide com  $y^k \bmod p$ . Para recuperar  $m$ , basta calcular:

- $m = c2 \cdot s^{-1} \bmod p$

O inverso modular  $s^{-1}$  é obtido com o algoritmo de Euclides estendido implementado na função `inverso_mod()`. Como  $p$  é primo, o inverso sempre existe.

#### 3.4 Tratamento de mensagens longas

Mensagens maiores que o módulo  $p$  são divididas em blocos de tamanho máximo  $(p.\text{bit\_length}() - 1) // 8$  bytes. Cada bloco é convertido para inteiro, cifrado individualmente e decifrado da mesma forma. Os blocos são reagrupados e a mensagem original é recuperada.

### 4. Implementação do Miller-Rabin

O teste de Miller-Rabin é um algoritmo probabilístico de primalidade. Para um candidato  $n$ , o teste baseia-se na decomposição  $n-1 = 2^s \cdot d$ , com  $d$  ímpar, e verifica se uma testemunha  $a$  se comporta como esperado para números primos.

O algoritmo implementado na função `eh_primo(n, dec)` segue os seguintes passos:

1. Casos base: retorna False para  $n < 2$  e para múltiplos de 2 diferentes de 2.
2. Decompõe  $n-1 = 2^s \cdot d$  dividindo sucessivamente por 2.
3. Para cada rodada: sorteia uma testemunha  $a$  aleatória no intervalo  $[2, n-2]$ .
4. Calcula  $x = a^d \bmod n$  via exponenciação modular eficiente.

5. Se  $x \neq 1$  e  $x \neq n-1$ , aplica  $s-1$  quadrações sucessivas em  $x \bmod n$ . Se nenhuma resultar em  $n-1$ , a testemunha prova que  $n$  é composto.
6. Repete o processo  $\text{dec} = 10$  vezes com testemunhas distintas.

Com 10 rodadas independentes, a probabilidade de um número composto passar no teste é inferior a  $4^{-(10)} \approx 10^{-6}$ . Na prática, para números da ordem de 256 bits, isso oferece segurança suficiente para a aplicação.

## 5. Principais Dificuldades

O desenvolvimento apresentou alguns desafios práticos que merecem registro:

- **Desempenho na geração de primos seguros:** A combinação de primos de Sophie Germain com o teste Miller-Rabin torna a geração lenta para chaves acima de 512 bits. Para 256 bits o tempo é aceitável (poucos segundos), mas escalar para tamanhos maiores exigiria otimizações adicionais, como usar gmpy2 para a aritmética modular.
- **Tratamento de blocos na codificação:** Garantir que cada bloco da mensagem fosse estritamente menor que  $p$  exigiu atenção no cálculo do tamanho máximo do bloco em bytes. Usar  $(p.\text{bit\_length}() - 1) // 8$  resolveu o problema, mas levou algum tempo de depuração para chegar à fórmula correta.
- **Decifração com blocos de tamanho fixo:** Na decodificação, cada bloco precisa ser convertido de volta para exatamente `block_size` bytes (com zero-padding à esquerda). Sem esse cuidado, bytes nulos intermediários entre blocos causam corrupção da mensagem ao concatenar.
- **Verificação da raiz primitiva em grupos grandes:** Para grupos muito grandes, iterar linearmente de  $g = 2$  em diante pode ser lento. No caso de primos seguros, porém, o primeiro gerador válido tende a aparecer cedo (geralmente  $g = 2$  ou um valor pequeno), o que torna a busca rápida na prática.

## 6. Instruções de Execução

### Pré-requisitos:

- Python 3.10 ou superior (o código usa sintaxe `tuple[int, int]` nos type hints)

### Execução padrão (modo interativo):

```
python elgamal_sc.py
```

O programa pedirá que você digite a mensagem a ser cifrada. Após a entrada, exibirá os parâmetros gerados, o texto cifrado e a mensagem decifrada por B.

### Execução não-interativa (stdin):

```
echo "mensagem secreta" | python elgamal_sc.py
```

Quando o stdin não é um terminal (pipe ou redirecionamento), o programa usa automaticamente a mensagem padrão de teste definida no código.

### Observação:

A geração do primo seguro de 256 bits pode levar entre 1 e 30 segundos dependendo da máquina.